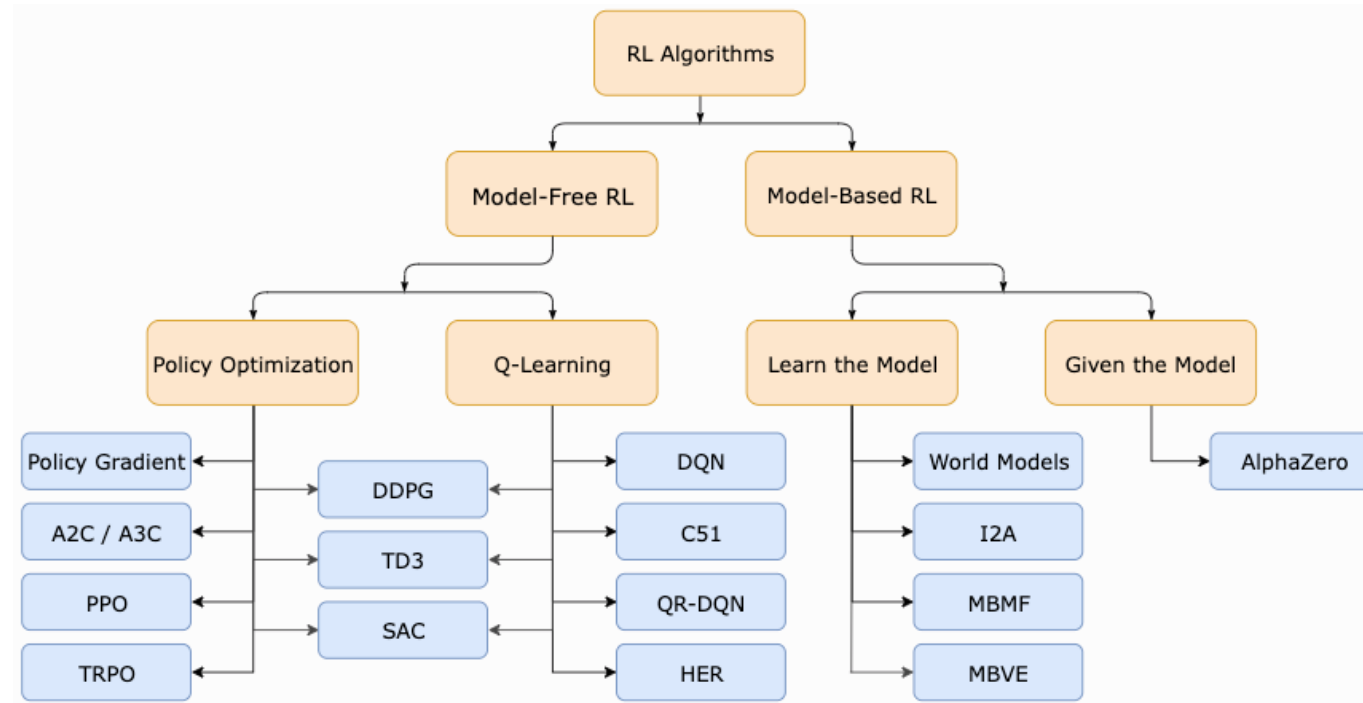


Advanced RL Algorithms



Taxonomy of RL algorithms, OpenAI

Instructor: Guanya Shi

Recording Reminder

- ☐ This session will be audio recorded for educational use by other students in this course.

Course Structure

Latest schedule: <https://16-831-s24.github.io/lectures>

- ☐ Topic group 1: Course introduction: What is robot learning and why?
- ☐ Topic group 2: Machine/Deep learning refresher
- ☐ Topic group 3: Imitation learning
- ☒ Topic group 4: Model-free reinforcement learning
- ☐ Topic group 5: Model-based reinforcement learning (one guest lecture)
- ☐ Topic group 6: Reinforcement learning from offline data (one guest lecture)
- ☐ Topic group 7: Bandit, preference-based learning, exploration
- ☐ Topic group 8: Specialized topics (one guest lecture)
- ☐ Project presentation & poster session

Outline

❑ Taxonomy of MFRL algorithms

- Recap: Q-learning
- Recap: Policy gradient
- Recap: Actor-critic
- Trade-off between different algorithms

❑ Advanced RL Algorithms:

- NPG: Natural policy gradient
- TRPO: Trust region policy optimization
- PPO: Proximal policy optimization
- Algorithms we have introduced: DQN and extensions, DDPG, SAC

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Outline

❑ Taxonomy of MFRL algorithms

- Recap: Q-learning
- Recap: Policy gradient
- Recap: Actor-critic
- Trade-off between different algorithms

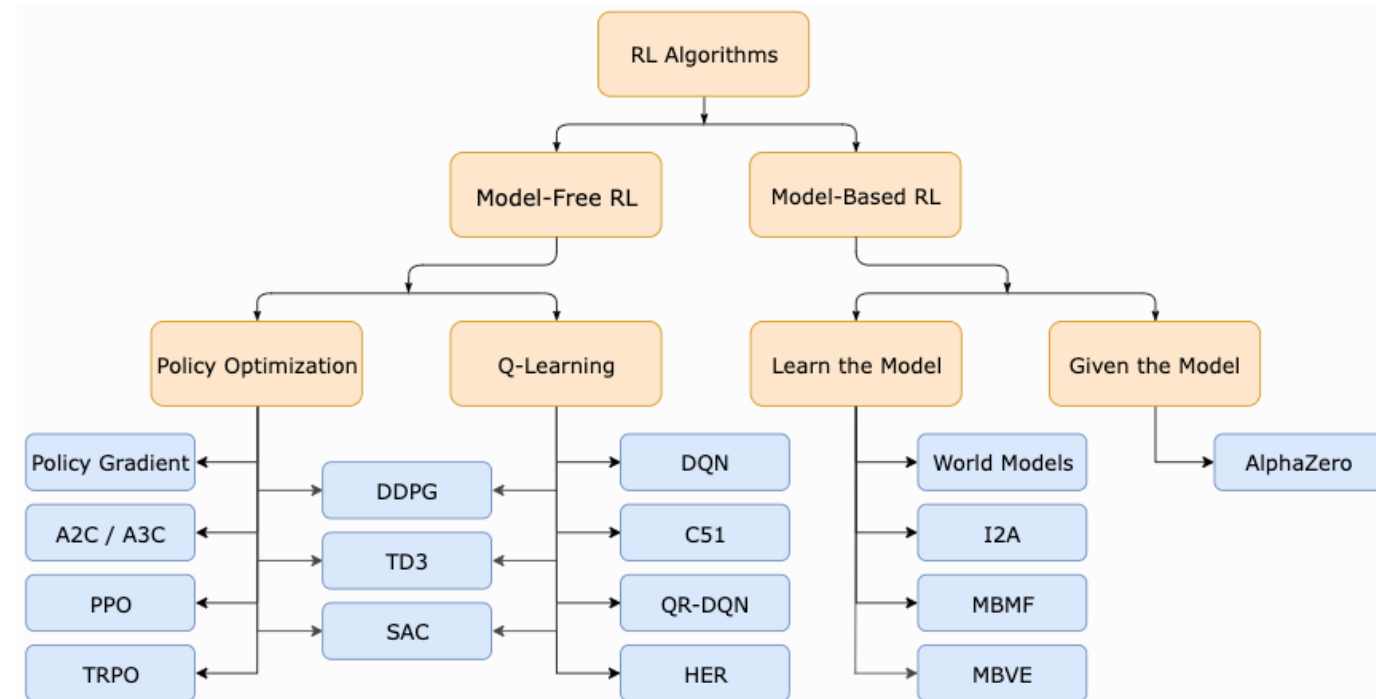
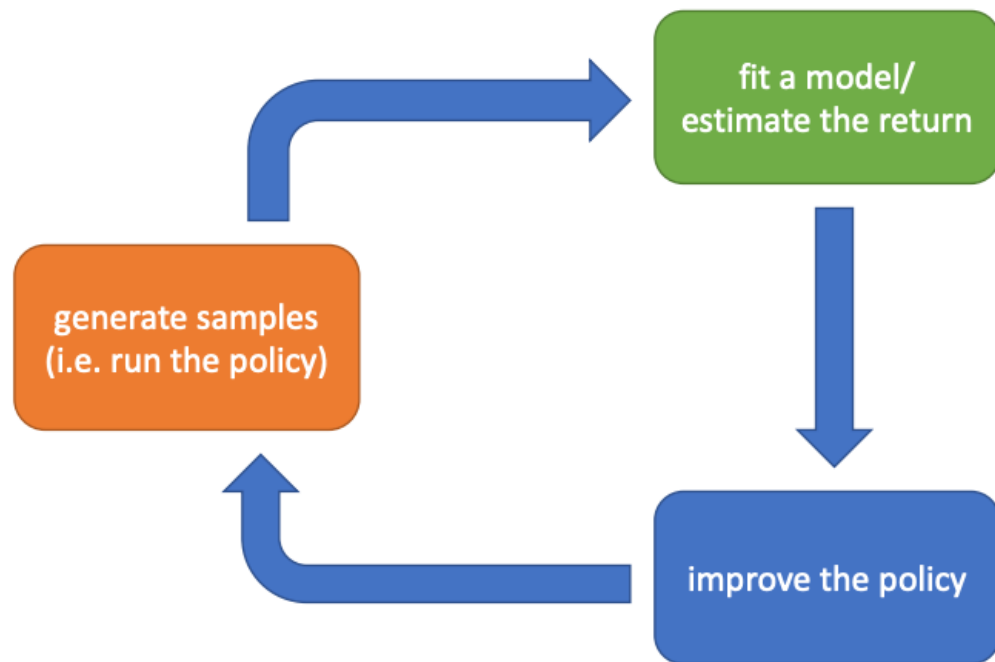
❑ Advanced RL Algorithms:

- NPG: Natural policy gradient
- TRPO: Trust region policy optimization
- PPO: Proximal policy optimization
- Algorithms we have introduced: DQN and extensions, DDPG, SAC

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

Taxonomy of MFRL Algorithms

- ❑ **Value-based** (example: FQI, DQN): estimate value function or Q-function of the optimal policy (no explicit policy).
- ❑ **Policy-gradient** (example: REINFORCE, NPG): directly optimize the policy
- ❑ **Actor-critic** (example: SAC, DDPG): estimate value function or Q-function of the current policy, use it to improve policy



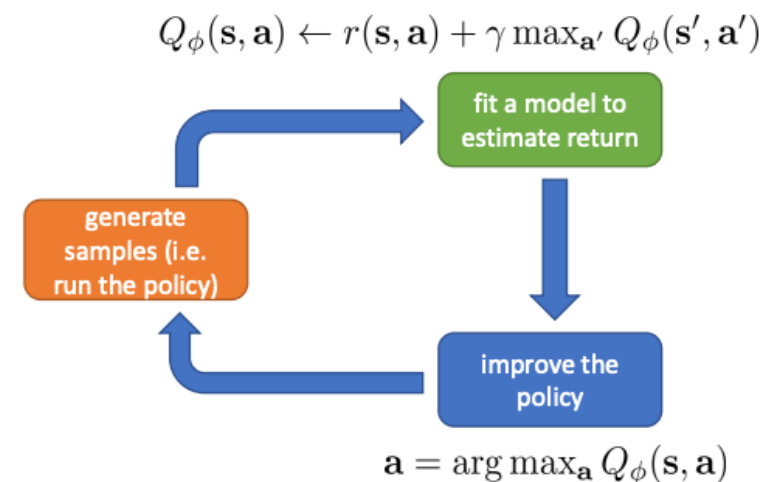
Recap: Q-Learning

□ Based on: **Bellman optimality equation**

$$Q^*(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \cdot \max_{a'} Q^*(s', a')]$$

□ Example: DQN (Deep Q-Network):

- Epsilon greedy policy gives a and observe (s, a, s', r)
- Update the replay buffer
- Sample a batch from the buffer. Compute the target $y_i = r(s_i, a_i) + \gamma \max_{a'_i} Q_{\phi^-}(s'_i, a'_i)$
- Update: $\phi \leftarrow \phi - \eta \sum_i (Q_{\phi}(s_i, a_i) - y_i) \nabla_{\phi} Q_{\phi}(s_i, a_i)$
- Periodically update ϕ^- : $\phi^- \leftarrow \phi$



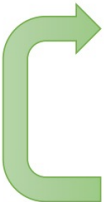
Recap: Policy Gradient

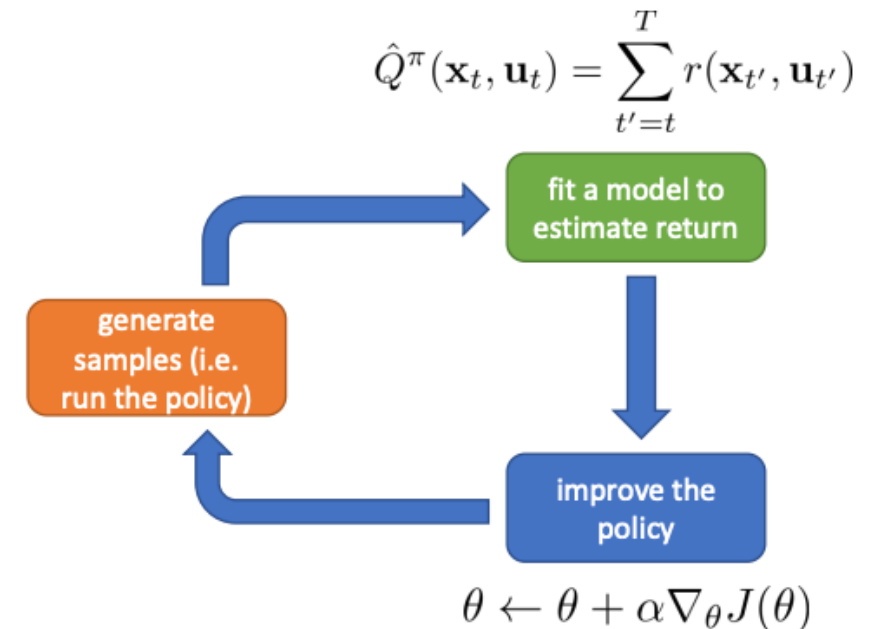
□ Based on: **Policy gradient theorem**

$$\underbrace{p_{\theta}(\mathbf{s}_1, \mathbf{a}_1, \dots, \mathbf{s}_T, \mathbf{a}_T)}_{p_{\theta}(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^T \underbrace{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t) p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)}_{J(\theta)}$$
$$\theta^* = \arg \max_{\theta} E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

$$\nabla_{\theta} J(\theta) = E_{\tau \sim p_{\theta}(\tau)} [R(\tau) \sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)]$$

□ Example: REINFORCE

- 
1. sample $\{\tau^i\}$ from $\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)$ (run the policy)
 2. $\nabla_{\theta} J(\theta) \approx \sum_i \left(\sum_t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t^i | \mathbf{s}_t^i) \right) \left(\sum_t r(\mathbf{s}_t^i, \mathbf{a}_t^i) \right)$
 3. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



Recap: Actor-Critic

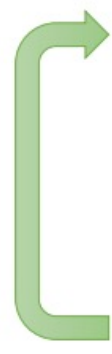
□ Based on: **Policy gradient with baselines**

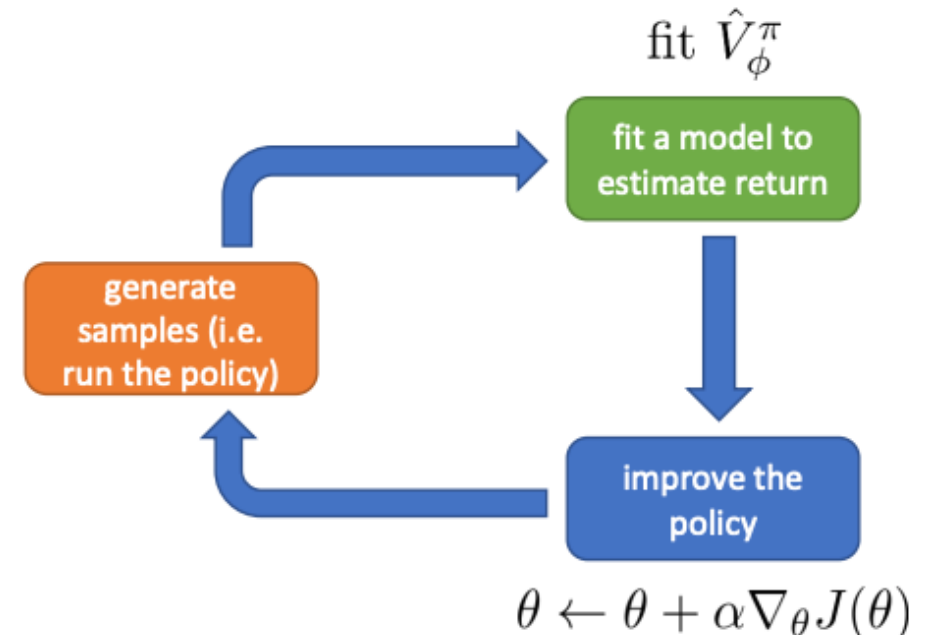
$$\nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [(Q^{\pi}(s, a) - V^{\pi}(s)) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

- Learn $V_{\phi}^{\pi}(s)$
- Estimate $A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$ by $r + \gamma V_{\phi}^{\pi}(s') - V_{\phi}^{\pi}(s)$

□ Example: online actor-critic

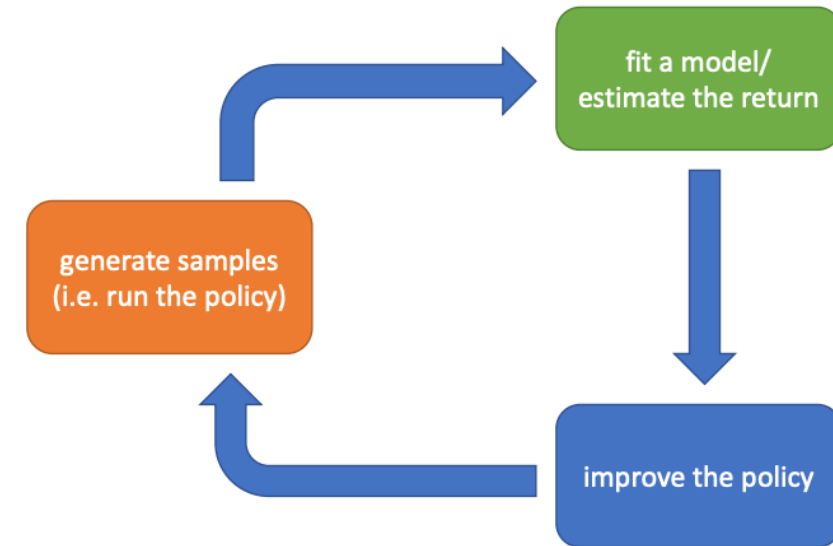
online actor-critic algorithm:

- 
1. take action $\mathbf{a} \sim \pi_{\theta}(\mathbf{a}|\mathbf{s})$, get $(\mathbf{s}, \mathbf{a}, \mathbf{s}', r)$
 2. update $\hat{V}_{\phi}^{\pi}$ using target $r + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}')$
 3. evaluate $\hat{A}^{\pi}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \hat{V}_{\phi}^{\pi}(\mathbf{s}') - \hat{V}_{\phi}^{\pi}(\mathbf{s})$
 4. $\nabla_{\theta} J(\theta) \approx \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \hat{A}^{\pi}(\mathbf{s}, \mathbf{a})$
 5. $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$



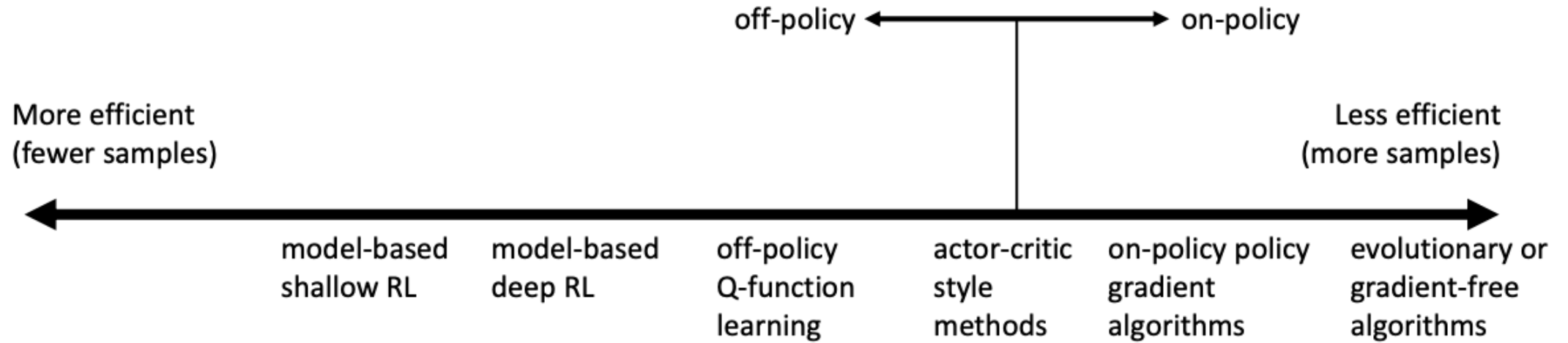
Trade-off Between Algorithms

- ❑ Why so many RL algorithms (even if not considering model-based RL)?
- ❑ Different tradeoffs! There is no “perfect” RL algorithm – depending on your applications
 - Sampling efficiency
 - Stability & ease of use
- ❑ Different assumptions
 - Stochastic / deterministic?
 - Continuous / discrete?
 - Finite horizon or infinite horizon?
- ❑ Different things are easy or hard in different settings
 - Learning value/policy/model: which one is easier?



Trade-off Between Algorithms

❑ Example: sampling efficiency



❑ This is only a general trend

❑ Warning: “more efficient” doesn’t necessarily mean shorter wall clock time!

- I.e., generating samples might be super fast (e.g., in simulation)

Outline

□ Taxonomy of MFRL algorithms

- Recap: Q-learning
- Recap: Policy gradient
- Recap: Actor-critic
- Trade-off between different algorithms

□ Advanced RL Algorithms:

- NPG: Natural policy gradient
- TRPO: Trust region policy optimization
- PPO: Proximal policy optimization
- Algorithms we have introduced: DQN and extensions, DDPG, SAC

(Some slides are from 22F, 23F, Berkeley CS 285, and CMU 10-703)

NPG (Natural Policy Gradient)

A Natural Policy Gradient

Sham Kakade

Gatsby Computational Neuroscience Unit
17 Queen Square, London, UK WC1N 3AR
<http://www.gatsby.ucl.ac.uk>
sham@gatsby.ucl.ac.uk

□ Recap: policy gradient

- Rollout π_θ . Compute $\nabla_\theta J(\theta)$
- Gradient ascent: $\theta' = \theta + \alpha \nabla_\theta J(\theta)$. Get a new policy $\pi_{\theta'}$

□ α controls the learning rate. Reformulate it to a constrained optimization problem:

$$\begin{aligned} \max_{\theta'} & (\theta' - \theta)^\top \nabla_\theta J(\theta) \\ \text{s. t. } & \|\theta - \theta'\|^2 \leq \epsilon \end{aligned}$$

□ ϵ controls the learning rate.

NPG (Natural Policy Gradient)

□ Recap: policy gradient

- Rollout π_θ . Compute $\nabla_\theta J(\theta)$
- Gradient ascent: $\theta' = \theta + \alpha \nabla_\theta J(\theta)$. Get a new policy $\pi_{\theta'}$

□ α controls the learning rate. Reformulate it to a constrained optimization problem:

$$\begin{aligned} \max_{\theta'} & (\theta' - \theta)^\top \nabla_\theta J(\theta) \\ \text{s.t. } & \|\theta - \theta'\|^2 \leq \epsilon \end{aligned}$$

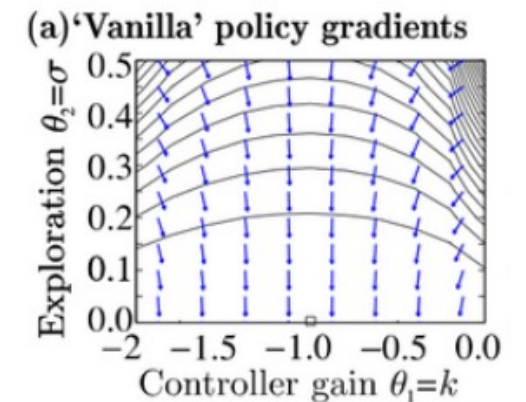
□ Problem: why is the constraint using 2-norm $\|\theta - \theta'\|^2$?

- Some part of θ might change π_θ a lot more than others!

□ Example:

$$r(\mathbf{s}_t, \mathbf{a}_t) = -\mathbf{s}_t^2 - \mathbf{a}_t^2$$

$$\log \pi_\theta(\mathbf{a}_t | \mathbf{s}_t) = -\frac{1}{2\sigma^2} (k\mathbf{s}_t - \mathbf{a}_t)^2 + \text{const} \quad \theta = (k, \sigma)$$



(image from Peters & Schaal 2008)

NPG (Natural Policy Gradient)

- ❑ Idea: Directly using some distance between two policies

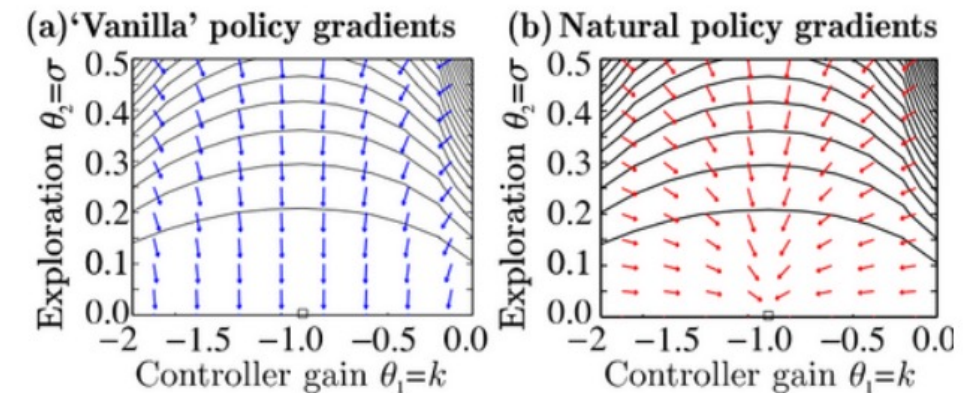
$$\begin{aligned} \max_{\theta'} & (\theta' - \theta)^\top \nabla_{\theta} J(\theta) \\ \text{s.t. } & D(\pi_{\theta}, \pi_{\theta'}) \leq \epsilon \end{aligned}$$

- ❑ Natural policy gradient: using Fisher-information matrix

$$\begin{aligned} \max_{\theta'} & (\theta' - \theta)^\top \nabla_{\theta} J(\theta) \\ \text{s.t. } & (\theta' - \theta)^\top \mathbf{F}(\theta' - \theta) \leq \epsilon \end{aligned}$$

$$\mathbf{F} = E_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s}) \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}|\mathbf{s})^T]$$

- ❑ The the update law becomes $\theta' = \theta + \alpha \mathbf{F}^{-1} \nabla_{\theta} J(\theta)$



(figure from Peters & Schaal 2008)

TRPO/PPO: Policy Gradient as Policy Iteration

□ Recap: policy gradient with value function as baselines

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t \gamma^t r(s_t, a_t) \right] \quad \nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [A^{\pi_{\theta}}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

□ A policy iteration perspective: $J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)} [\sum_t \gamma^t A^{\pi_{\theta}}(s_t, a_t)]$

□ Proof:

$$\begin{aligned} J(\theta') - J(\theta) &= J(\theta') - E_{\mathbf{s}_0 \sim p(\mathbf{s}_0)} [V^{\pi_{\theta}}(\mathbf{s}_0)] \\ &= J(\theta') - E_{\tau \sim p_{\theta'}(\tau)} [V^{\pi_{\theta}}(\mathbf{s}_0)] \\ &= J(\theta') - E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t V^{\pi_{\theta}}(\mathbf{s}_t) - \sum_{t=1}^{\infty} \gamma^t V^{\pi_{\theta}}(\mathbf{s}_t) \right] \\ &= J(\theta') + E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_{\theta}}(\mathbf{s}_{t+1}) - V^{\pi_{\theta}}(\mathbf{s}_t)) \right] \\ &= E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right] + E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (\gamma V^{\pi_{\theta}}(\mathbf{s}_{t+1}) - V^{\pi_{\theta}}(\mathbf{s}_t)) \right] \\ &= E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t (r(\mathbf{s}_t, \mathbf{a}_t) + \gamma V^{\pi_{\theta}}(\mathbf{s}_{t+1}) - V^{\pi_{\theta}}(\mathbf{s}_t)) \right] \\ &= E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \end{aligned}$$

TRPO/PPO: Policy Gradient as Policy Iteration

- Recap: policy gradient with value function as baselines

$$J(\theta) = E_{\tau \sim p_{\theta}(\tau)} \left[\sum_t \gamma^t r(s_t, a_t) \right] \quad \nabla_{\theta} J(\theta) \propto E_{s \sim p_{\theta}, a \sim \pi_{\theta}} [A^{\pi_{\theta}}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

- A policy iteration perspective:

$$J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right]$$

- Why it could be useful? We can directly do $\max_{\theta'} J(\theta') - J(\theta)$ to improve the policy!
- Why this form is not that useful? Chicken and egg problem! We cannot compute $E_{\tau \sim p_{\theta'}(\tau)}[\cdot]$ without knowing θ'
- **The key idea of TRPO/PPO:** control the distance between θ' and θ such that we can use $E_{\tau \sim p_{\theta}(\tau)}[\cdot]$ to estimate $J(\theta') - J(\theta)$!

TRPO/PPO: Policy Gradient as Policy Iteration

□ A policy iteration perspective:

$$J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t) \right]$$

□ Using importance sampling again:

$$\begin{aligned} E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(s_t, \mathbf{a}_t) \right] &= \sum_t E_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)} \left[\gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \\ &= \sum_t E_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \end{aligned}$$

□ Can we use $s_t \sim p_\theta(s_t)$ to approximate?

□ Yes! If $D(\pi_\theta, \pi_{\theta'})$ is small.

TRPO: Trust Region Policy Optimization

□ A policy iteration perspective:

$$J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t) \right]$$

□ TRPO: If $D(\pi_\theta, \pi_{\theta'})$ is small, we have:

$$\sum_t E_{\mathbf{s}_t \sim p_{\theta'}(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right] \approx \sum_t E_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

□ Therefore:

$$\theta' \leftarrow \arg \max_{\theta'} \sum_t E_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

such that $D_{\text{KL}}(\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) \| \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)) \leq \epsilon$

Trust Region Policy Optimization

John Schulman
Sergey Levine
Philipp Moritz
Michael Jordan
Pieter Abbeel

University of California, Berkeley, Department of Electrical Engineering and Computer Sciences

JOSCHU@EECS.BERKELEY.EDU
SLEVINE@EECS.BERKELEY.EDU
PCMORITZ@EECS.BERKELEY.EDU
JORDAN@CS.BERKELEY.EDU
PABBEEL@CS.BERKELEY.EDU

PPO: Proximal Policy Optimization

- ❑ A policy iteration perspective:

$$J(\theta') - J(\theta) = E_{\tau \sim p_{\theta'}(\tau)} \left[\sum_t \gamma^t A^{\pi_\theta}(s_t, a_t) \right]$$

- ❑ The objective in TRPO:

$$\theta' \leftarrow \arg \max_{\theta'} \sum_t E_{\mathbf{s}_t \sim p_\theta(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_\theta}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

$$\text{such that } D_{\text{KL}}(\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t) \| \pi_\theta(\mathbf{a}_t | \mathbf{s}_t)) \leq \epsilon$$

- ❑ It is a constrained non-convex optimization problem!
 - Complicated and hard/slow to solve
- ❑ PPO: design a surrogate objective function

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov

OpenAI

{joschu, filip, prafula, alec, oleg}@openai.com

PPO: Proximal Policy Optimization

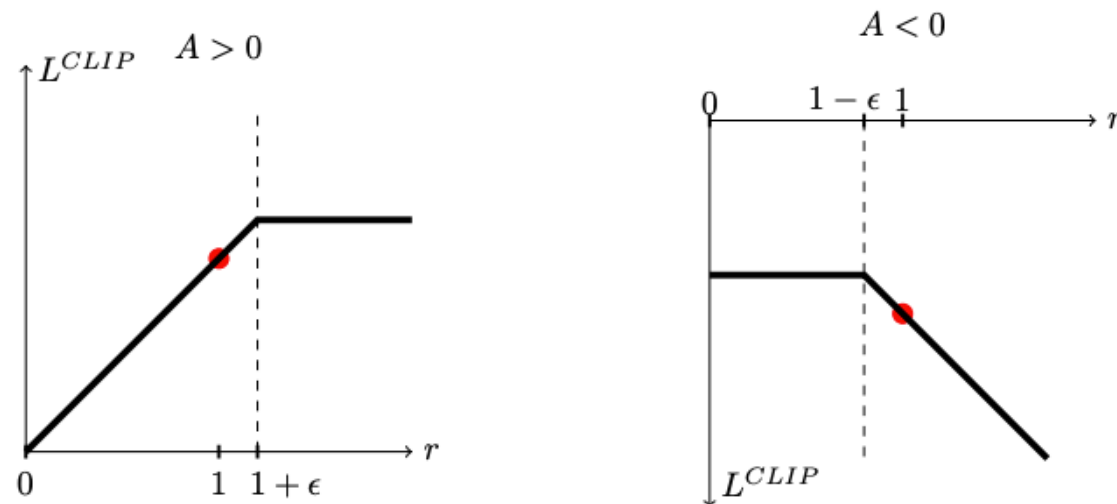
□ The objective function in TRPO:

$$\theta' \leftarrow \arg \max_{\theta'} \sum_t E_{\mathbf{s}_t \sim p_{\theta}(\mathbf{s}_t)} \left[E_{\mathbf{a}_t \sim \pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \left[\frac{\pi_{\theta'}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)} \gamma^t A^{\pi_{\theta}}(\mathbf{s}_t, \mathbf{a}_t) \right] \right]$$

□ PPO: design a surrogate objective function.

□ Define $r_t(\theta') = \frac{\pi_{\theta'}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)}$. Change the term in the red circle to ($\epsilon = 0.2$ works well):

$$L_t^{CLIP}(\theta') = \min(r_t(\theta') \gamma^t A^{\pi_{\theta}}(s_t, a_t), \text{clip}(r_t(\theta'), 1 - \epsilon, 1 + \epsilon) \gamma^t A^{\pi_{\theta}}(s_t, a_t))$$



PPO: Proximal Policy Optimization

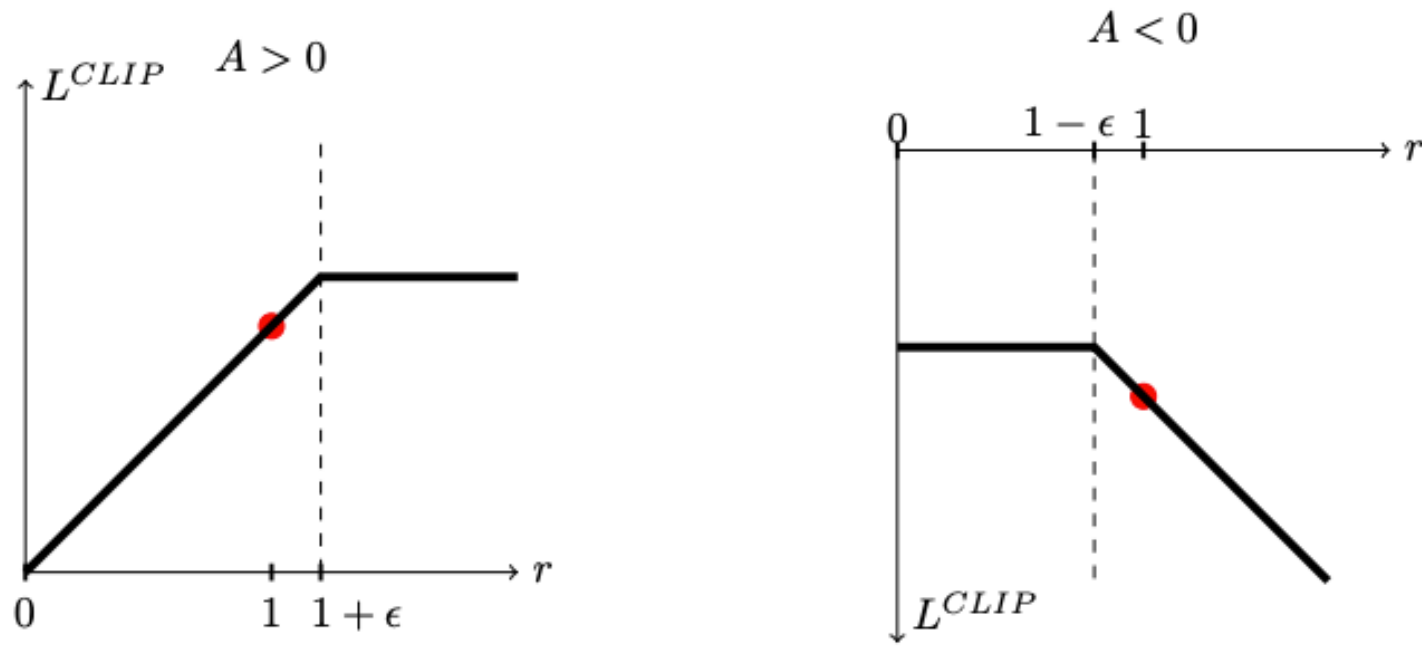
□ Define $r_t(\theta') = \frac{\pi_{\theta'}(a_t|s_t)}{\pi_{\theta}(a_t|s_t)}$. Change the term in the red circle to ($\delta = 0.2$ works well):

$$L_t^{CLIP}(\theta') = \min(r_t(\theta')\gamma^t A^{\pi_{\theta}}(s_t, a_t), \text{clip}(r_t(\theta'), 1 - \delta, 1 + \delta)\gamma^t A^{\pi_{\theta}}(s_t, a_t))$$

□ Note that $L_t^{CLIP}(\theta') \leq r_t(\theta')\gamma^t A^{\pi_{\theta}}(s_t, a_t)$. I.e., a pessimistic lower bound!

□ If we only have the clip part ($\text{clip}(r_t(\theta'), 1 - \delta, 1 + \delta)\gamma^t A^{\pi_{\theta}}(s_t, a_t)$):

- “Very bad” $r_t(\theta')$ will NOT be punished (e.g., $r_t(\theta') \ll 1$ when $A > 0$)!



PPO in Robot Learning

Agile But Safe: Learning Collision-Free High-Speed Legged Locomotion

Tairan He^{1†} Chong Zhang^{2†} Wenli Xiao¹ Guanqi He¹ Changliu Liu¹ Guanya Shi¹

¹Carnegie Mellon University ²ETH Zürich

[†]Equal Contributions <https://agile-but-safe.github.io>

RMA: Rapid Motor Adaptation for Legged Robots

Ashish Kumar
UC Berkeley

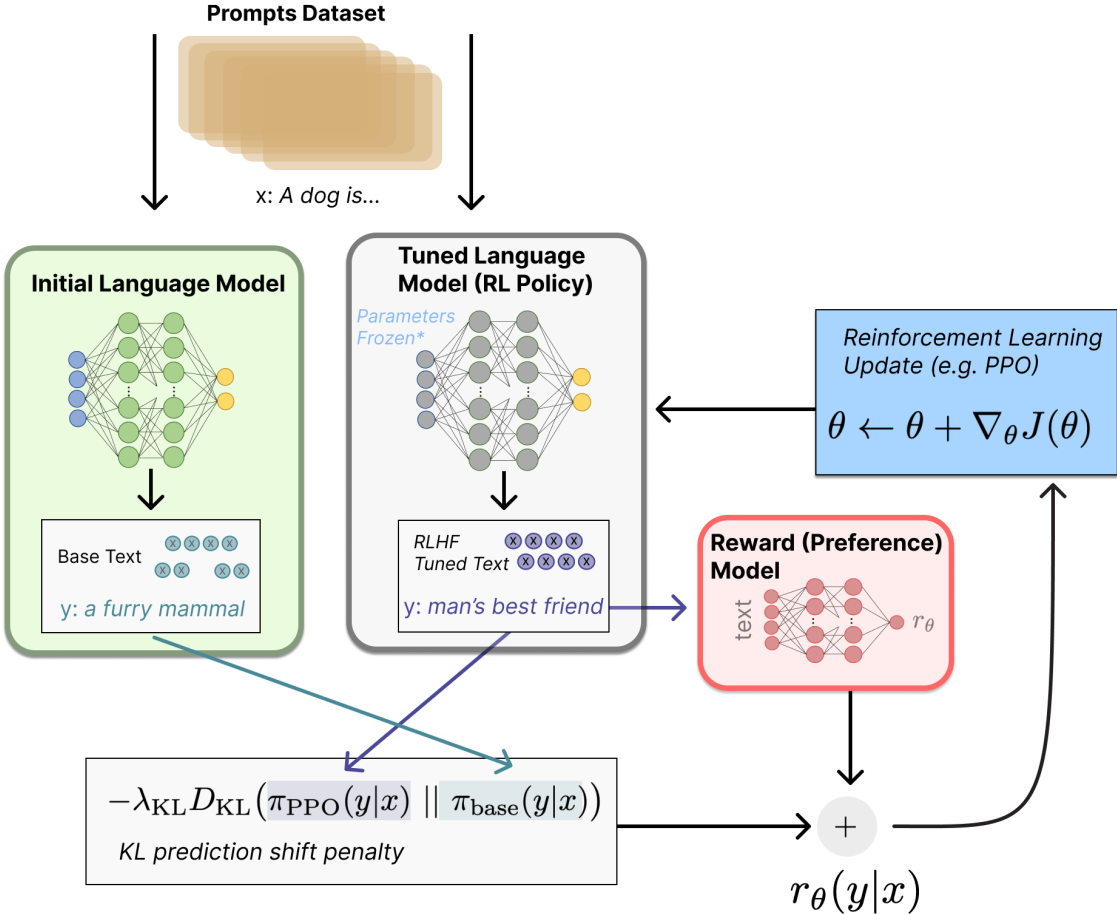
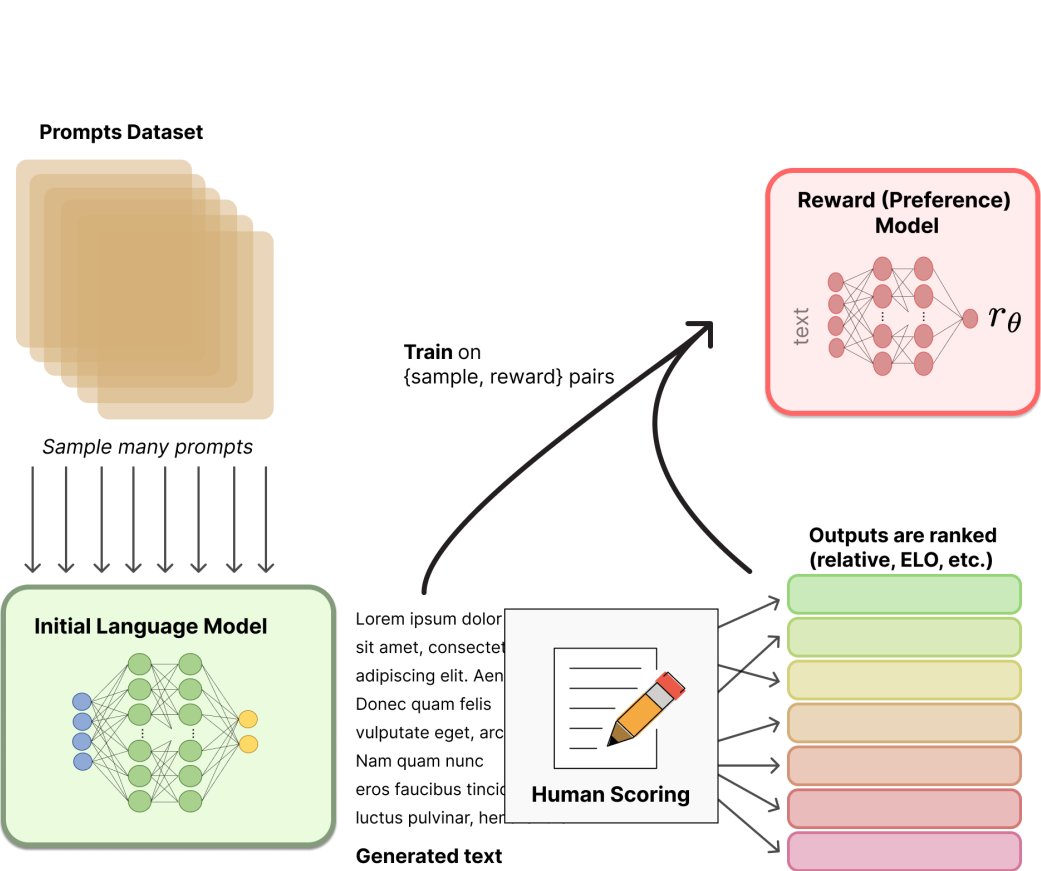
Zipeng Fu
Carnegie Mellon University

Deepak Pathak
Carnegie Mellon University

Jitendra Malik
UC Berkeley, Facebook



PPO in RLHF for LLM (GPTs)



<https://huggingface.co/blog/rlhf>

Summary

- ❑ Advanced **model-free** & **online** RL algorithms (algorithms people are using for robot learning) we have covered:
 - DQN and extensions (DDQN)
 - DDPG
 - SAC
 - NPG
 - TRPO
 - PPO
- ❑ Other algorithms:
 - A3C/A2C: for parallel training
 - D4PG (distributional DDPG)
 - MADDPG: multi-agent DDPG
 - ...
- ❑ The best way to learn them is to implement them yourself!

Thank You!